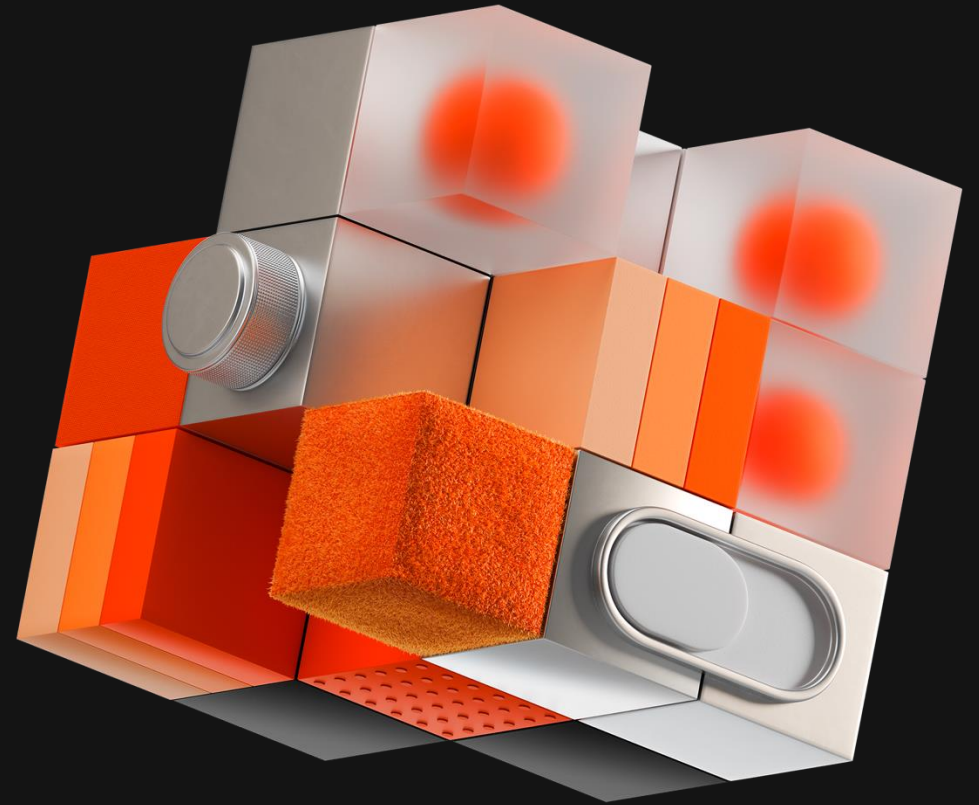


Интеграция с другими приложениями

REST-клиенты, gRPC, Spring и OpenAPI

НЕДЕЛЯ 19



Наш план на сегодня



Интеграция как контракт



RestTemplate и RestClient



WebClient и reactive I/O



gRPC в Java/Spring

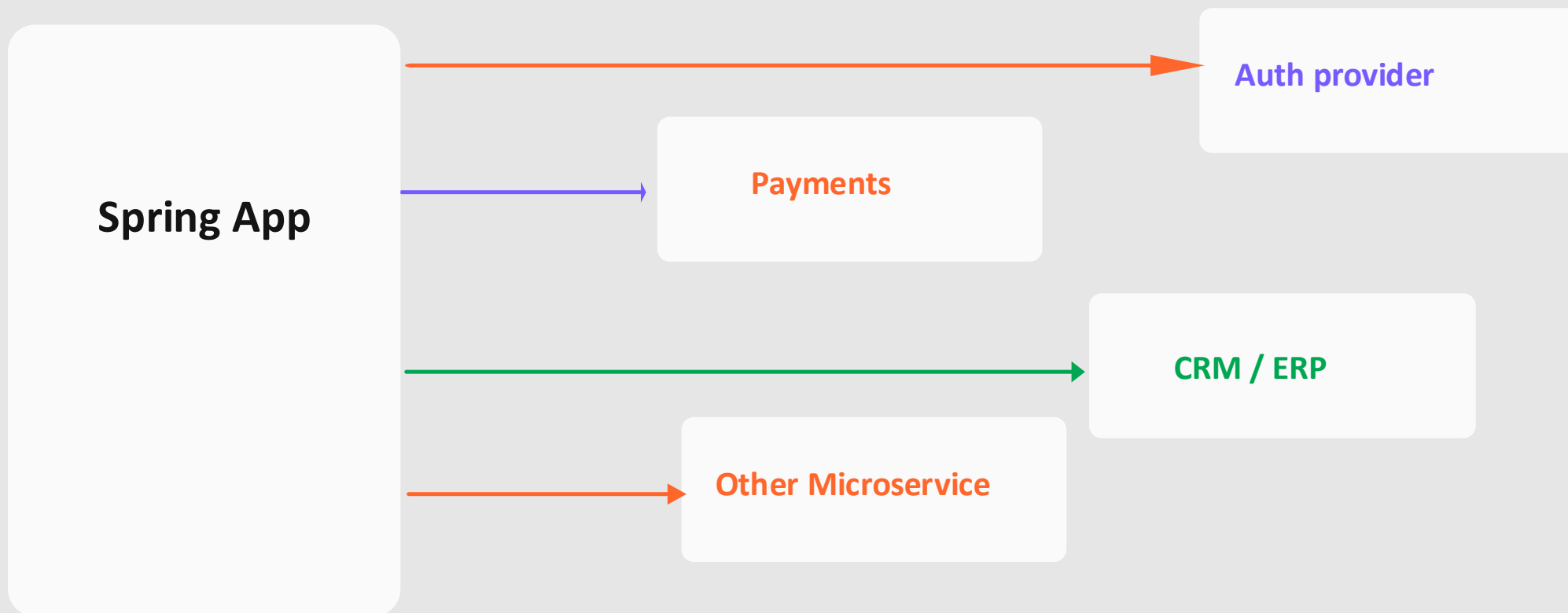


Spring и OpenAPI



Где появляются интеграции

Spring-приложение редко живет одно: оно вызывает чужие API и само предоставляет API для других сервисов.



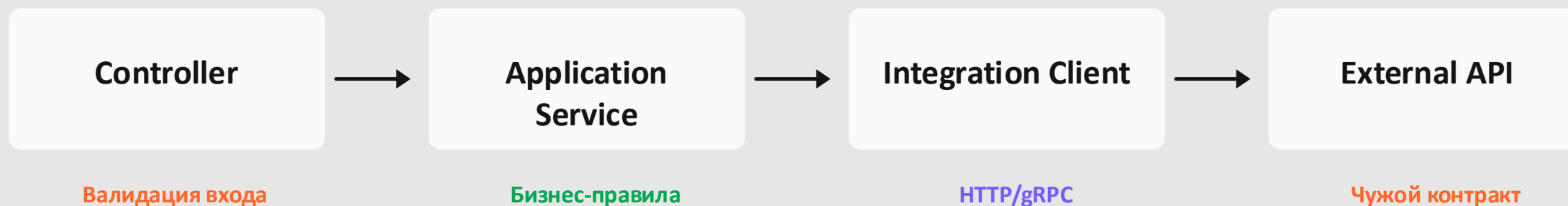
Интеграция — это место, где ваш код встречается чужую доступность, чужой контракт и чужой темп изменений.

Сам запрос сделать легко, главное – остальное

- URI, DTO, статусы, схемы
- timeout или deadline
- бизнесовые и технические ошибки
- logs, metrics, tracing

Слой клиента изолирует HTTP-детали

Контроллер и бизнес-сервис не должны знать, как именно устроен внешний вызов.



Правило: HTTP/gRPC-клиент — отдельный адаптер. Внутри бизнес-кода остаются доменные методы, а не строки URL и статусы.

Что входит в контракт вызова

Request

method, path, headers, body

Response

status, body, error format

Time budget

timeout, retry, backoff

Semantics

idempotency, rate limit, ownership

Выбор клиента начинается со сценария

Инструмент	Модель	Когда уместен
RestTemplate	blocking	поддержка существующего кода
RestClient	blocking	новый sync-код в Spring MVC
WebClient	non-blocking	WebFlux, streaming, reactive composition
gRPC	HTTP/2 RPC	внутренние service-to-service API

REST-клиенты в Spring

Один HTTP API можно вызвать разными моделями

RestTemplate: знакомый template-подход

```
CourseDto course = restTemplate.getForObject(  
    "/api/courses/{id}",  
    CourseDto.class,  
    "spring-rest"  
);
```

- Синхронный вызов:
текущий поток ждет ответ.
- API хорошо знаком по
legacy Spring-проектам.
- Много overload-методов:
удобно быстро, сложнее
читать в больших клиентах.

RestTemplate: конфигурация важнее вызова

```
@Bean
RestTemplate courseRestTemplate(RestTemplateBuilder builder) {
    return builder
        .rootUri(properties.baseUrl())
        .setConnectTimeout(Duration.ofSeconds(2))
        .setReadTimeout(Duration.ofSeconds(5))
        .errorHandler(new CourseApiErrorHandler())
        .build();
}
```

- Base URL убирает дублирование.
- Connect/read timeout задают бюджет ожидания.
- ErrorHandler переводит HTTP-ошибки в доменную ошибку клиента.

RestTemplate и generic-типы

```
ResponseEntity<List<CourseDto>> response =  
    restTemplate.exchange(  
        "/api/courses",  
        HttpMethod.GET,  
        null,  
        new ParameterizedTypeReference<List<CourseDto>>() {}  
    );
```

- Из-за type erasure одного List.class недостаточно.
- exchange дает полный контроль над методом, headers и типом ответа.
- Цена контроля — более шумный код.

HTTP-ошибки — тоже часть контракта

Код клиента должен различать бизнесовые ответы и технические сбои.

400

validation

Retry не требуется

401/403

auth

Средиректить на доступ

404

not found

Может быть бизнес-сценарием

409

conflict

решить состояние

429

rate limit

backoff

5xx

temporary failure

Возможно требуется retry

RestClient

Современный синхронный клиент с fluent API



RestClient: тот же sync-вызов, но лаконичнее

```
CourseDto course = restClient  
    .get()  
    .uri("/api/courses/{id}", "spring-rest")  
    .retrieve()  
    .body(CourseDto.class);
```

- По модели выполнения это blocking-клиент.
- Цепочка читается как HTTP-запрос.
- Хороший default для нового Spring MVC-кода.

RestClient: POST и статусы рядом с запросом

```
EnrollmentResponse response = restClient
    .post()
    .uri("/api/enrollments")
    .body(request)
    .retrieve()
    .onStatus(HttpStatusCode::isError, (req, res) -> {
        throw CourseApiException.from(res);
    })
    .body(EnrollmentResponse.class);
```

- Тело запроса и обработка ответа находятся в одной цепочке.
- `onStatus` удобен для особой логики конкретного вызова.
- `defaultStatusHandler` — для общей политики клиента.

Миграция RestTemplate → RestClient

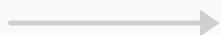
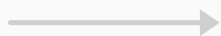
RestTemplate

`getForObject`

`postForObject`

`exchange`

`ResponseErrorHandler`



RestClient

`get().uri().retrieve().body()`

`post().uri().body().retrieve().body()`

fluent request chain

`defaultStatusHandler / onStatus`

WebClient

Reactive Stream и все все все



WebClient: запрос становится pipeline

Mono и Flux описывают будущий результат. Выполнение начинается при подписке.



Blocking-код ждет результат в потоке. Non-blocking pipeline продолжает работу, когда результат готов.

WebClient: GET возвращает Mono

```
Mono<CourseDto> course = webClient.get()
    .uri("/api/courses/{id}", "spring-rest")
    .retrieve()
    .bodyToMono(CourseDto.class);
```

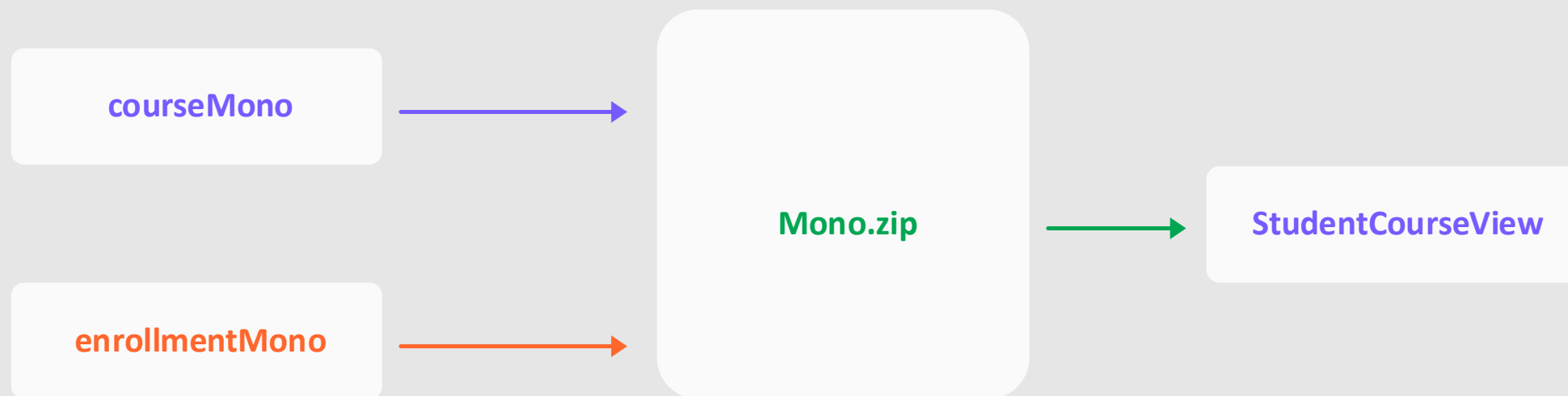
- Это еще не CourseDto, а описание асинхронного результата.
- Mono — 0 или 1 значение.
- Код удобен, когда дальше есть reactive-композиция.

WebClient: ошибки остаются в pipeline

```
.onStatus(HttpStatusCode::isError, response ->
    response.bodyToMono(String.class)
        .map(body -> new CourseApiException(
            response.statusCode().value(),
            body
        ))
)
```

- Ошибка становится сигналом в reactive-цепочке.
- Можно читать тело ошибки без блокировки.
- Важно не прятать технические сбои под null или Optional.

Reactive-композиция: несколько вызовов как один сценарий



```
Mono.zip(courseMono, enrollmentMono)
  .map(tuple -> new StudentCourseView(
    tuple.getT1(),
    tuple.getT2()
  ));
```

WebClient полезен для потоков событий

```
Flux<NotificationEvent> events = webClient.get()  
    .uri("/api/notifications")  
    .retrieve()  
    .bodyToFlux(NotificationEvent.class)  
    .take(3);
```



event 1



event 2



event 3

- Flux — поток 0..N значений.
- Подходит для SSE, long polling, streaming JSON.
- Не нужно держать один поток JVM заблокированным на ожидании каждого события.

.block()

Если весь WebClient-код заканчивается **.block()**, возможно, сценарий на самом деле синхронный.

- Синхронный сценарий – это тоже нормально
- Стоит заранее выбирать: реактивный подход или синхронный

Что стоит настроить

 **timeout / deadline**

 **backoff и rate limit**

 **correlation ID**

 **маскирование секретов**

 **retry только для безопасных операций**

 **circuit breaker**

 **метрики latency/error rate**

 **контрактные тесты**

gRPC в Java/Spring

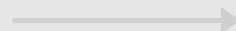
Контракт через .proto и generated stubs



REST и gRPC описывают разные формы API

REST / OpenAPI

- resources + HTTP methods
- JSON
- хорошо читается руками
- удобно для публичных API



gRPC / Protobuf

- service + rpc methods
- binary Protobuf
- generated stubs
- часто для внутренних API

Оба варианта требуют явного контракта, версионирования и политики ошибок.

.proto — контракт gRPC-сервиса

```
service CourseCatalogService {  
  rpc GetCourse(GetCourseRequest) returns (Course) {}  
  rpc ListCourses(ListCoursesRequest)  
    returns (ListCoursesResponse) {}  
  rpc WatchCourseEvents(WatchCourseEventsRequest)  
    returns (stream CourseEvent) {}  
}
```

- service и rpc методы — центральная модель API.
- Номера полей в message являются частью бинарного контракта.
- Java-код клиента и сервера генерируется из .proto.

gRPC server в Spring: сервис как bean

```
@Service
class CourseCatalogGrpcService
    extends CourseCatalogServiceGrpc
        .CourseCatalogServiceImplBase {

    @Override
    public void getCourse(GetCourseRequest request,
        StreamObserver<Course> response) {
        // map domain result to protobuf response
    }
}
```

- Spring управляет зависимостями сервиса.
- gRPC runtime вызывает generated base class.
- Бизнес-репозиторий лучше держать отдельно от protobuf-маппинга.

gRPC client: channel, stub, deadline

```
ManagedChannel channel = ManagedChannelBuilder
    .forAddress(host, port)
    .usePlaintext()
    .build();

CourseCatalogServiceBlockingStub stub =
    CourseCatalogServiceGrpc.newBlockingStub(channel);

Course course = stub
    .withDeadlineAfter(2, TimeUnit.SECONDS)
    .getCourse(request);
```

- Channel — соединение и транспортная настройка.
- Stub — типобезопасный клиентский API.
- Deadline задает общий лимит времени на RPC.

Типы gRPC-вызовов

Unary

request → response

Server streaming

request → response*

Client streaming

request* → response

Bidi streaming

request* ↔ response*

gRPC-ошибки: не HTTP-статусы, а Status

Клиент должен понимать коды gRPC и не превращать все в RuntimeException без контекста.

NOT_FOUND

ресурс не найден

может быть бизнес-ответом

INVALID_ARGUMENT

ошибка входа

не retry

UNAVAILABLE

сервис недоступен

retry с backoff

DEADLINE_EXCEEDED

бюджет истек

проверить timeout

PERMISSION_DENIED

нет доступа

исправить auth

INTERNAL

ошибка сервера

логировать correlation

Когда gRPC уместен

Хорошо подходит

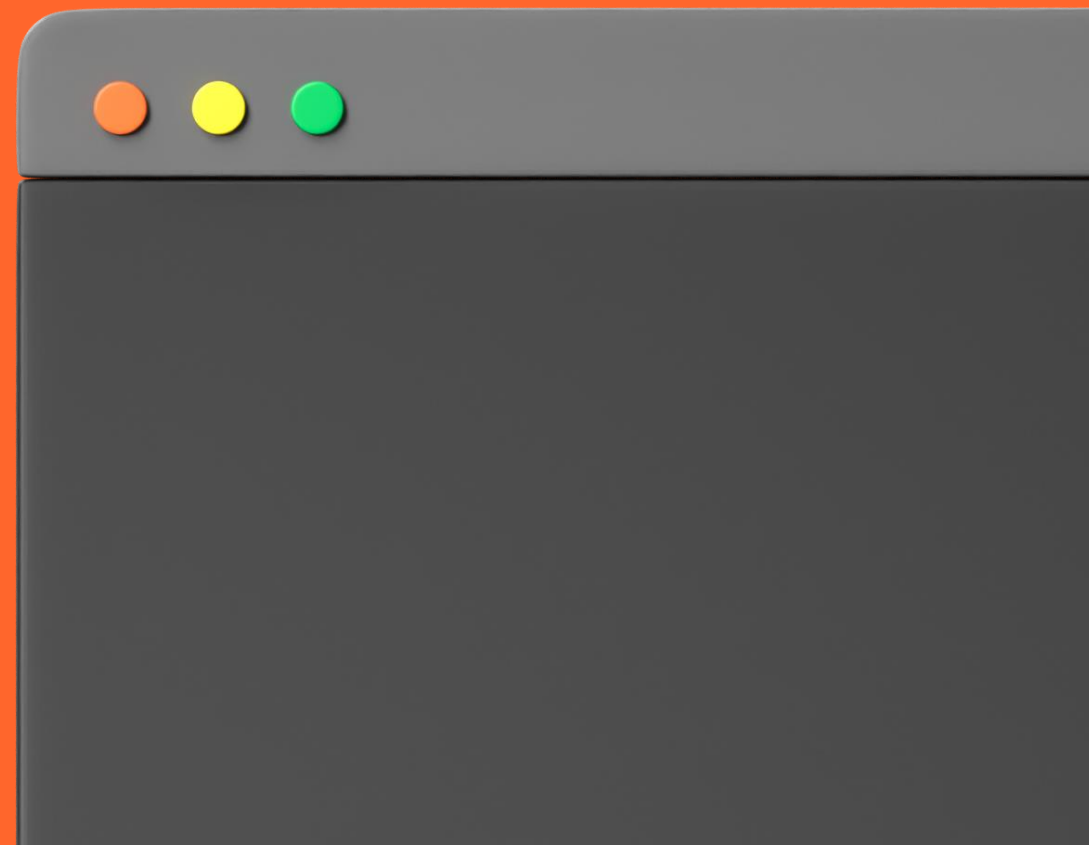
- внутренние service-to-service API
- строгий контракт и генерация кода
- низкая цена сериализации
- server streaming или bidi streaming

Осторожнее

- публичный browser-facing API
- ручная отладка через curl
- сервисы без общей схемы релизов
- нужен простой JSON для внешних клиентов

Spring и OpenAPI

REST-контракт как машинно-читаемый артефакт



Swagger UI — это витрина, OpenAPI — контракт



springdoc-openapi maven dependency

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>
    springdoc-openapi-starter-webmvc-ui
  </artifactId>
</dependency>
```

- /v3/api-docs — JSON-контракт.
- /swagger-ui.html — интерактивная страница поверх контракта.
- Схемы выводятся из controller, DTO, Jackson и validation-аннотаций.

Аннотации уточняют смысл API

```
@Operation(summary = "Получить курс")
@ApiResponse(responseCode = "200",
    description = "Курс найден")
@ApiResponse(responseCode = "404",
    description = "Курс не найден")
@GetMapping("/api/courses/{id}")
CourseDto getCourse(@PathVariable String id) { ... }
```

- OpenAPI должен описывать не только happy path.
- DTO-примеры помогают клиентам быстрее понять формат.
- Validation-аннотации становятся частью схемы.

Code-first и contract-first

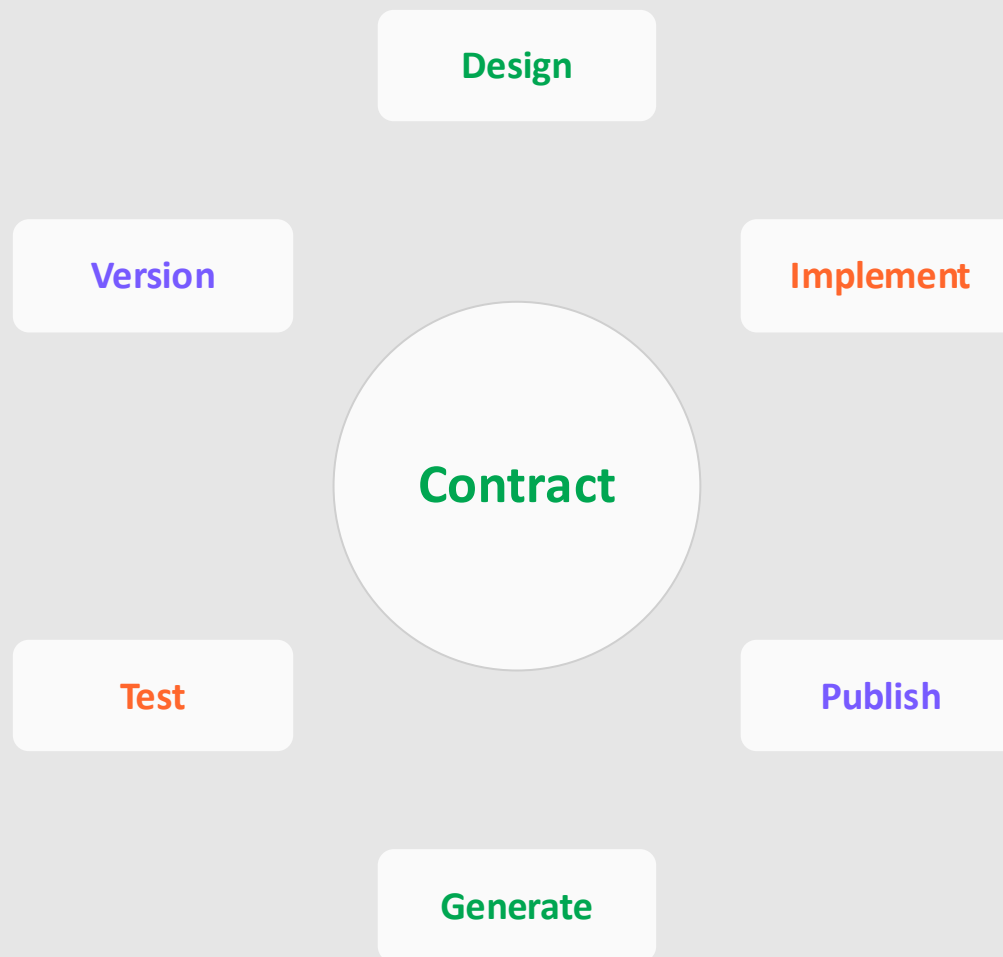
Code-first

- Быстро стартовать из Spring-кода
- Хорошо для внутренних API
- Контракт следует за реализацией

Contract-first

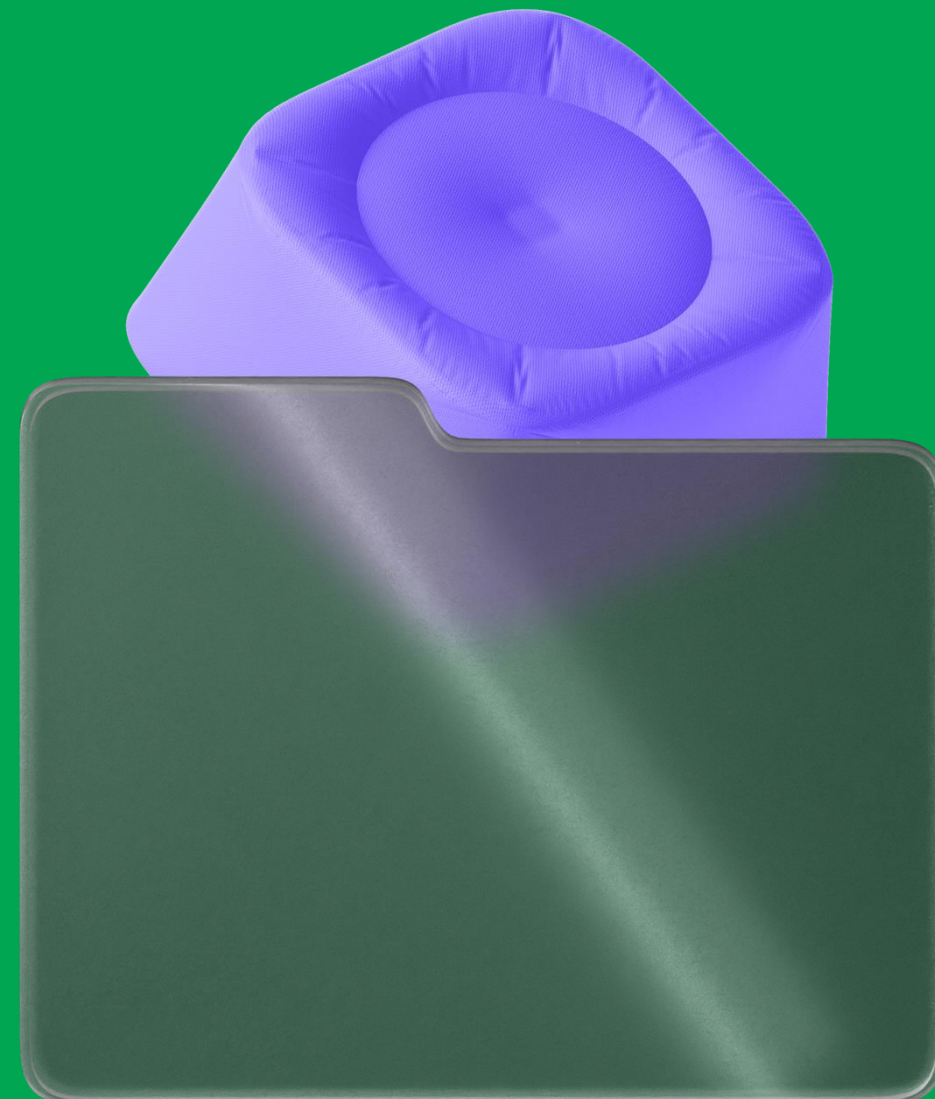
- Контракт согласуется до кода
- Удобно для независимых групп
- Клиенты и тесты генерируются заранее

Жизненный цикл контракта



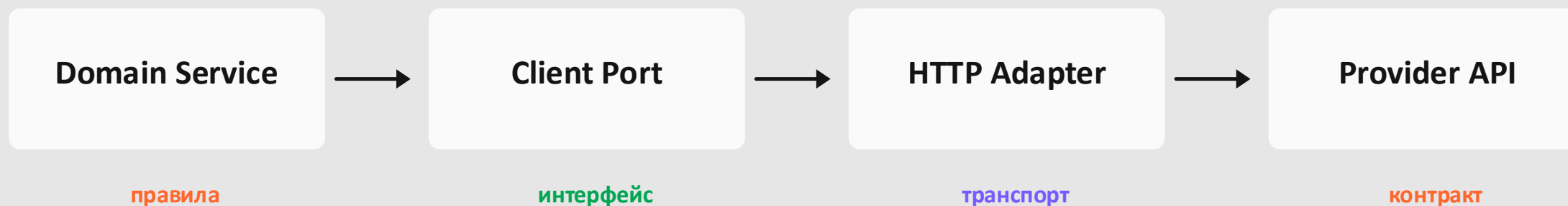
Интеграционный слой

Как сделать клиент читаемым и сопровождаемым



Порт и адаптер для внешнего API

Бизнес-слой зависит от интерфейса, а HTTP/gRPC детали остаются во внешнем адаптере.



Правило: HTTP/gRPC-клиент — отдельный адаптер. Внутри бизнес-кода остаются доменные методы, а не строки URL и статусы.

Интерфейс клиента говорит языком домена

```
public interface CourseProviderClient {  
    CourseDto findCourse(String courseId);  
  
    EnrollmentResponse enroll(  
        EnrollmentRequest request);  
  
    List<CourseDto> findActiveCourses();  
}
```

- Внутри нет HTTP method и URL.
- Интерфейс легко заменить в тестах.
- Transport-specific ошибки мапятся ниже.

Configuration properties отделяют код от окружения

```
@ConfigurationProperties("course.provider")
public record ProviderProperties(
    URI baseUrl,
    Duration connectTimeout,
    Duration responseTimeout,
    String apiKey
) {
}
```

- Адреса и бюджеты времени не хардкодятся.
- Разные окружения получают разные значения.
- Секреты не попадают в исходный код.

Семантика ошибки важнее номера

Validation

клиент отправил неверные данные

Not found

ресурс отсутствует или скрыт

Conflict

состояние изменилось

Unavailable

сервис временно недоступен



Какие ошибки можно ретраить

GET + 503

обычно можно

backoff + limit

POST + timeout

опасно

нужен ключ

400

нельзя

исправить request

401

нельзя

обновить auth

429

можно позже

respect retry-after

409

зависит

решить конфликт

Idempotency-Key делает POST безопаснее

```
String key = commandId.toString();

restClient.post()
    .uri("/api/enrollments")
    .header("Idempotency-Key", key)
    .body(request)
    .retrieve()
    .body(EnrollmentResponse.class);
```

- Одинаковый ключ означает одну бизнес-команду.
- Provider должен хранить результат первого выполнения.
- Без поддержки provider-a header не магический.

Чтение ProblemDetail на клиенте

```
ProblemDetail problem = objectMapper.readValue(  
    body, ProblemDetail.class);  
  
throw new CourseApiException(  
    response.getStatusCode().value(),  
    problem.getTitle(),  
    problem.getDetail()  
);
```

- Ошибка сохраняет технический status.
- Title/detail помогают диагностировать причину.
- Не теряйте тело ответа при маппинге.

DTO mapping: где проходит граница

Слой	Тип данных	Причина
Controller	public API DTO	контракт приложения
Service	domain model	правила предметной области
Integration	provider DTO	чужой контракт
Persistence	entity	схема хранения

Review новой интеграции

 есть client interface

 есть timeout

 нет URL в сервисах

 есть trace

 есть properties

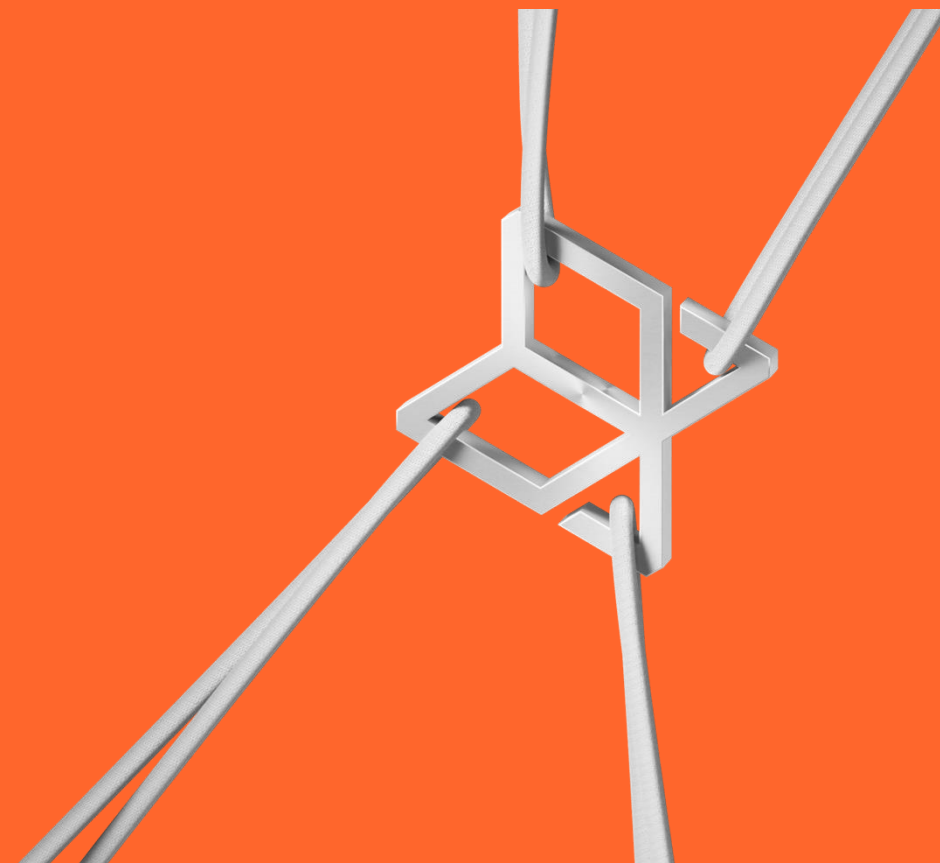
 описаны ошибки

 есть tests

 есть contract source

RestTemplate подробнее

Что важно знать при поддержке существующего
кода



URI лучше собирать безопасно

```
URI uri = UriComponentsBuilder
    .fromPath("/api/courses")
    .QueryParam("status", "ACTIVE")
    .build()
    .toUri();

CourseDto[] courses = restTemplate
    .getForObject(uri, CourseDto[].class);
```

- Не склеивайте query string вручную.
- Builder корректно кодирует параметры.
- Так проще добавлять optional filters.

ResponseEntity нужен, когда важен status

```
ResponseEntity<CourseDto> response =  
    restTemplate.getForEntity(  
        "/api/courses/{id}",  
        CourseDto.class,  
        id  
    );  
  
HttpStatusCode status = response.getStatusCode();
```

- Иногда body недостаточно.
- Headers могут содержать rate limit или pagination.
- Status помогает различить сценарии.

RequestEntity делает запрос явным

```
RequestEntity<EnrollmentRequest> request =  
    RequestEntity.post("/api/enrollments")  
        .accept(MediaType.APPLICATION_JSON)  
        .contentType(MediaType.APPLICATION_JSON)  
        .body(enrollment);  
  
restTemplate.exchange(request,  
    EnrollmentResponse.class);
```

- Method, headers и body собраны вместе.
- Удобно для сложных запросов.
- Но fluent API RestClient обычно читается легче.

Не создавайте RestTemplate в каждом методе

```
// плохо
RestTemplate restTemplate = new RestTemplate();

// лучше
@RequiredArgsConstructor
class CourseClient {
    private final RestTemplate restTemplate;
}
```

- Bean можно централизованно настроить.
- Проще тестировать через mock server.
- Настройки не размазываются по коду.

RestTemplate: сильные и слабые стороны

Сильные стороны

- понятная синхронная модель
- много существующего кода
- хорошая поддержка Spring Boot

Недостатки

- монструозные overload-методы
- легко потерять единую policy
- для нового sync-кода есть RestClient

RestClient подробнее

Fluent API для императивных сервисов



Default headers в одном месте

```
RestClient client = RestClient.builder()  
    .baseUrl(properties.baseUrl().toString())  
    .defaultHeader(HttpHeaders.USER_AGENT,  
        "course-service")  
    .defaultHeader(HttpHeaders.ACCEPT,  
        MediaType.APPLICATION_JSON_VALUE)  
    .build();
```

- Повторяемые headers задаются один раз.
- Код запроса остается короче.
- Auth лучше подключать через interceptor/filter.

Per-request headers остаются рядом с вызовом

```
restClient.get()  
    .uri("/api/courses/{id}", id)  
    .header("X-Request-Source", "lecture")  
    .retrieve()  
    .body(CourseDto.class);
```

- Локальные headers видны в контексте запроса.
- Так удобно передавать feature flags.
- Общие headers не надо дублировать.

RestClient может вернуть ResponseEntity

```
ResponseEntity<CourseDto> response =  
    restClient.get()  
        .uri("/api/courses/{id}", id)  
        .retrieve()  
        .toEntity(CourseDto.class);  
  
CourseDto body = response.getBody();
```

- Нужен status или headers — берите entity.
- Happy path остается компактным.
- Для нестандартной логики есть exchange.

Единый exception для provider-a

```
public class CourseApiException
    extends RuntimeException {
    private final int status;
    private final String responseBody;

    static CourseApiException from(
        ClientHttpResponse response) {
        // read status and body
    }
}
```

- Исключение хранит важный контекст.
- Вызывающий слой не зависит от HTTP library.
- Такая ошибка легче логируется и тестируется.

План миграции на RestClient

 найти клиентов

 выделить properties

 перенести один endpoint

 обновить тесты

 покрыть ошибки

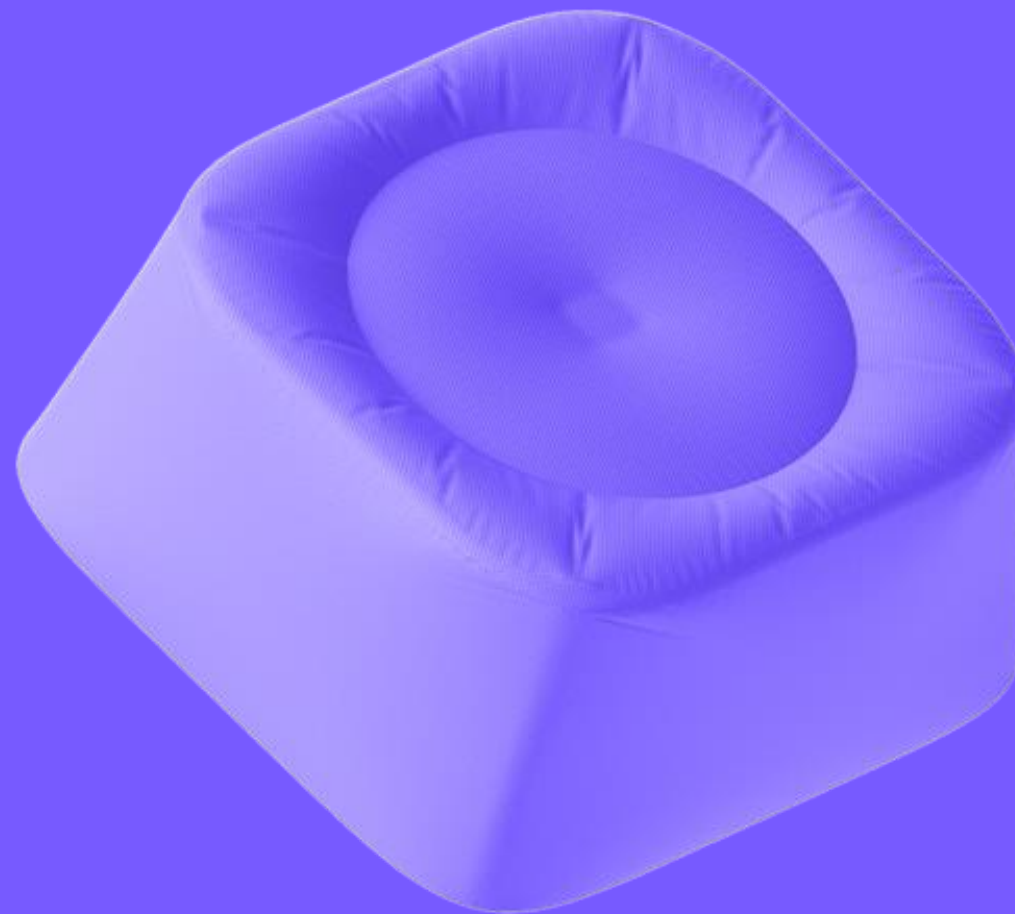
 согласовать exception

 сравнить логи

 удалить дубли

WebClient подробнее

Когда reactive модель действительно помогает



WebClient builder похож, но модель другая

```
WebClient client = WebClient.builder()
    .baseUrl(properties.baseUrl().toString())
    .defaultHeader(HttpHeaders.ACCEPT,
        MediaType.APPLICATION_JSON_VALUE)
    .build();
```

- Синтаксис похож на RestClient.
- Возвращаемые типы — Mono и Flux.
- Сходство API не делает код синхронным.

Filter добавляет cross-cutting behavior

```
ExchangeFilterFunction traceFilter =  
    (request, next) -> {  
        ClientRequest traced = ClientRequest  
            .from(request)  
            .header("X-Correlation-Id", traceId())  
            .build();  
        return next.exchange(traced);  
    };
```

- Filter похож на interceptor.
- Работает внутри reactive цепочки.
- Удобен для trace, auth и метрик.

flatMap строит зависимые вызовы

```
Mono<StudentCourseView> view =  
    courseClient.findCourse(courseId)  
        .flatMap(course ->  
            enrollmentClient.enroll(course.id())  
                .map(enrollment ->  
                    new StudentCourseView(  
                        course, enrollment)));
```

- Второй вызов зависит от результата первого.
- flatMap разворачивает вложенный Mono.
- Так не нужно блокировать поток между шагами.

flatMap concurrency ограничивает нагрузку

```
Flux.fromIterable(courseIds)
    .flatMap(
        id -> courseClient.findCourse(id),
        8
    )
    .collectList();
```

- Параллелизм должен быть ограничен.
- Иначе легко перегрузить provider API.
- Это reactive-версия bulkhead thinking.

Mono и CompletableFuture отличаются

CompletableFuture

- Future обычно eager
- одно значение
- нет встроенного backpressure

Mono / Flux

- Mono/Flux lazy до подписки
- 0..N значений через Flux
- операторы композиции и сигналов

WebClient: частые ошибки

 block в WebFlux

 retry без фильтра

 unbounded flatMap

 мешаем модели

 нет timeout

 не читаем body ошибки

 теряем context

 путаем Mono и value

gRPC

подробнее

Схема, stubs, deadline и versioning



.proto: package и Java options

```
syntax = "proto3";  
  
package course.catalog.v1;  
  
option java_package =  
    "edu.course.grpc.catalog.v1";  
option java_multiple_files = true;
```

- package задает пространство имен контракта.
- Java options управляют generated code.
- Версия часто отражается в package.

Enum в Protobuf начинается с UNKNOWN

```
enum CourseStatus {  
  COURSE_STATUS_UNSPECIFIED = 0;  
  COURSE_STATUS_ACTIVE = 1;  
  COURSE_STATUS_ARCHIVED = 2;  
}
```

- Нулевое значение используется по умолчанию.
- UNSPECIFIED помогает отличить отсутствие значения.
- Не переименовывайте смысл старых значений.

StatusRuntimeException надо маппить

```
try {  
    return stub.getCourse(request);  
} catch (StatusRuntimeException ex) {  
    if (ex.getStatus().getCode()  
        == Status.Code.NOT_FOUND) {  
        throw new CourseNotFoundException(id);  
    }  
    throw new CourseGrpcException(ex);  
}
```

- gRPC status — часть контракта.
- Бизнес-слой не должен знать StatusRuntimeException.
- Сохраняйте cause и metadata для диагностики.

Metadata передает auth и trace

```
Metadata metadata = new Metadata();
metadata.put(AUTHORIZATION,
    "Bearer " + token);
metadata.put(CORRELATION_ID, traceId);

ClientInterceptor interceptor =
    MetadataUtils
        .newAttachHeadersInterceptor(metadata);
```

- Metadata — аналог headers в gRPC.
- Auth и trace можно добавлять interceptor-ом.
- Не логируйте metadata с секретами целиком.

Channel нужно закрывать аккуратно

```
@PreDestroy
void shutdown() throws InterruptedException {
    channel.shutdown();
    if (!channel.awaitTermination(
        5, TimeUnit.SECONDS)) {
        channel.shutdownNow();
    }
}
```

- Channel держит ресурсы транспорта.
- Приложение должно завершаться чисто.
- В Spring это удобно спрятать в bean lifecycle.

gRPC review checklist

 есть deadline

 metadata безопасна

 stream cancellation

 proto package version

 статусы маппятся

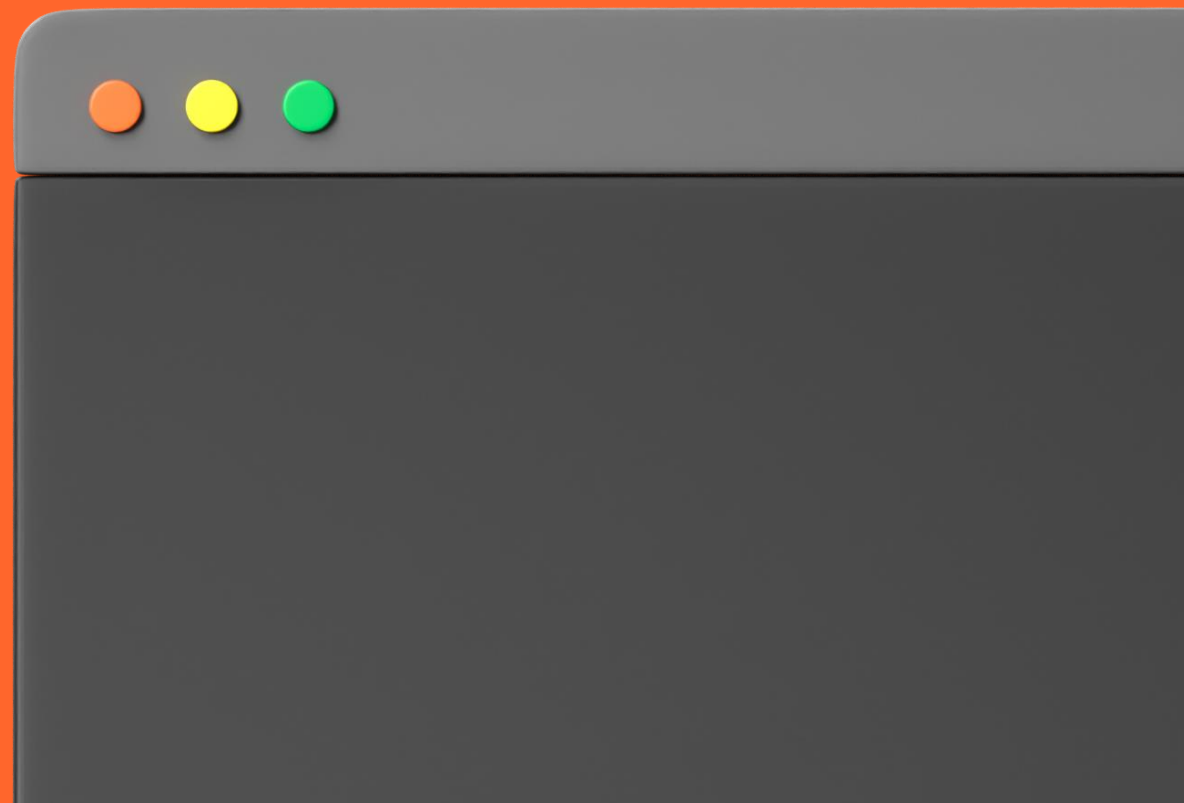
 reserved поля

 channel lifecycle

 contract tests

OpenAPI подробнее

Контракт, review и совместимость изменений



OpenAPI YAML: paths и schemas

```
paths:
  /api/courses/{id}:
    get:
      operationId: getCourse
      parameters:
        - name: id
          in: path
          required: true
      responses:
        '200':
          description: Course found
```

- paths описывает операции API.
- operationId должен быть стабильным.
- responses должны отражать реальные статусы.

operationId важен для генерации

```
@Operation(  
    operationId = "getCourse",  
    summary = "Получить курс"  
)  
@GetMapping("/api/courses/{id}")  
CourseDto getCourse(@PathVariable String id)
```

- Генераторы используют operationId как имя метода.
- Случайные имена усложняют клиентский код.
- Менять operationId — breaking change для generated clients.

Generated client не отменяет review

```
CourseApi api = new CourseApi(client);  
CourseDto course = api.getCourse("spring-rest");  
  
// wrapper still maps provider errors  
return mapper.toDomain(course);
```

- Generated code лучше не смешивать с доменом.
- Поверх него нужен тонкий adapter.
- Ошибки и retry policy остаются вашей ответственностью.

Совместимость изменений REST API

Изменение	Совместимость	Комментарий
добавить optional field	обычно да	клиенты игнорируют
удалить field	нет	ломает парсинг/логику
новый 4xx	частично	обновить клиентов
переименовать path	нет	новая версия
добавить endpoint	да	не ломает старых

API review перед публикацией

 clear names

 examples

 auth documented

 rate limits

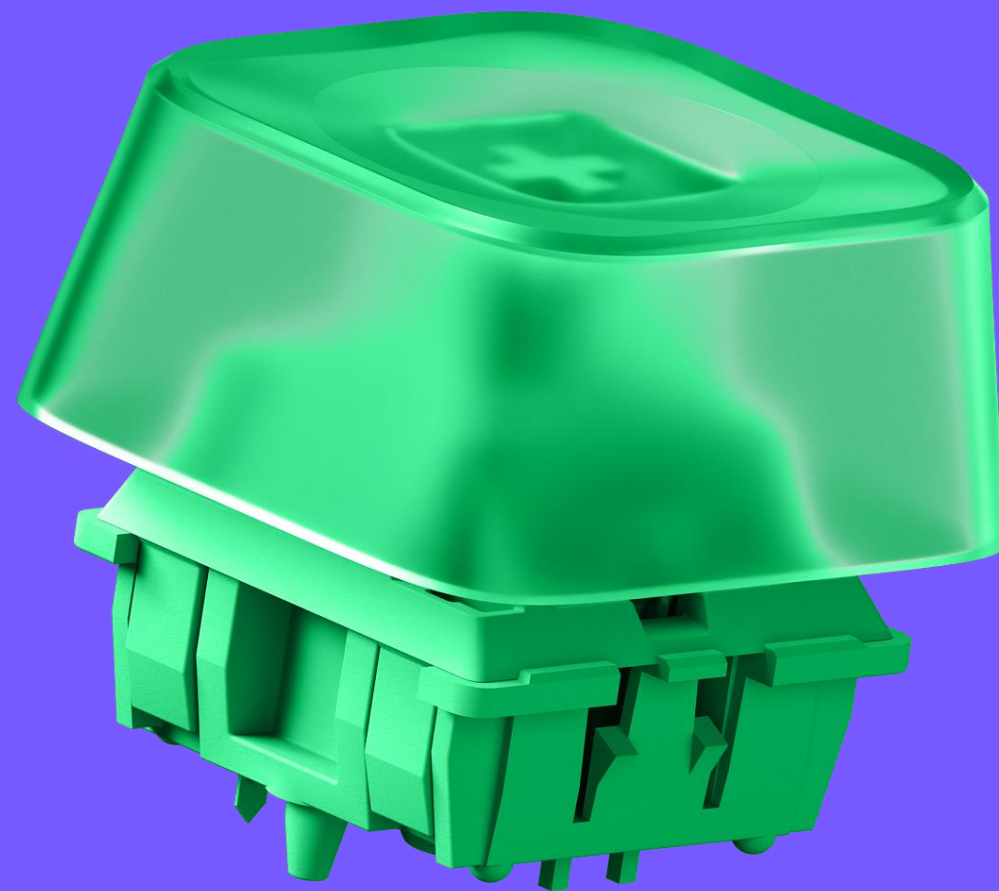
 stable operationId

 error schema

 pagination

 compatibility note

Так что и когда
выбрать?



Выбор по сценарию

Сценарий	Инструмент	Почему
MVC вызывает JSON API	RestClient	sync и читаемо
старый клиент стабилен	RestTemplate	минимум риска
WebFlux + SSE	WebClient	Flux stream
internal low-latency RPC	gRPC	typed protobuf
публичный REST API	OpenAPI	контракт для клиентов

Симптомы плохой интеграции

Эти признаки обычно означают, что проблема не в библиотеке, а в дизайне слоя клиента.

URL везде

нет adapter

выделить client

нет timeout

зависания

здать budget

catch Exception

теряем смысл

typed errors

retry all

дубли

idempotency

no contract

ломается внезапно

OpenAPI/proto

no trace

сложно чинить

correlation

Один фасад может скрывать транспорт

```
@Service
@RequiredArgsConstructor
class CourseCatalog {
    private final CourseProviderClient client;

    CourseCard loadCard(String id) {
        CourseDto dto = client.findCourse(id);
        return mapper.toCard(dto);
    }
}
```

- Сервис не знает, RestClient это или gRPC.
- Транспорт можно заменить за интерфейсом.
- Тесты бизнес-логики становятся проще.

Sync facade и reactive facade

Sync facade

- `CourseDto findCourse(id)`
- проще для MVC
- ошибка через exception

Reactive facade

- `Mono<CourseDto> findCourse(id)`
- естественно для WebFlux
- ошибка как signal

Практические задания для студентов

 добавить endpoint

 описать 404

 написать RestClient метод

 сделать WebClient stream

 добавить deadline

 расширить .proto

 обновить OpenAPI

 написать contract test

Итоговые инженерные правила

Хорошая интеграция скучна в эксплуатации: она быстро падает, понятно логируется и не удивляет контрактом.

- явная граница клиента
- короткие time budget
- ошибки сохраняют контекст
- контракт проверяется автоматически

Чеклист

 Понятный согласованный interface

 timeout задан

 ошибки типизированы

 контракт рядом

 нет URL вне adapter

 retry объяснен

 trace проходит

 есть тест сбоя

Спасибо за внимание!

Вопросы

